

End-to-End System Blueprint

AI-Based Automatic Block Planning · Chennai Division

Purpose The engineering contract — how every component works and fits together

Companion documents 2. Product Requirements & Data Specification · 3. Complete Project Compendium

Pilot scope Chennai Division, Southern Railway

Departments Engineering (P.Way) · Traction Distribution · Signal & Telecommunication

Version 2.0 — consolidated after full design review

What this system does. Three railway departments independently request track access for maintenance through BDMS. Nobody merges those requests, so a corridor that could be taken once for three hours is taken three times. This system pools all three departments' pending work, scores it by risk, matches it against real corridor availability from COA, packs compatible co-located jobs into shared blocks, routes the result through human approval, and re-plans when reality breaks the schedule.

Contents

- 1 Scope, boundaries and design principles
- 2 The network model
- 3 System architecture
- 4 The processing pipeline, stage by stage
- 5 Stage 3 in detail: window enumeration
- 6 Stage 4 in detail: clustering and merge candidates
- 7 Stage 5 in detail: the optimiser
- 8 The four machine learning models
- 9 Multi-horizon planning
- 10 Emergency re-planning
- 11 Approval workflow and BDMS handoff
- 12 Evaluation harness
- 13 Configuration parameter reference
- 14 Technology stack

1 Scope, boundaries and design principles

1.1 What is in scope

Maintenance of **fixed infrastructure** requiring temporary withdrawal of the line or its equipment from service, across three departments:

Department	Assets	Source system
Engineering (P.Way)	Track, rails, sleepers, ballast, bridges, points & crossings	Track Management System (TMS)
Traction Distribution	OHE, masts, contact wire, insulators, substations	Traction Distribution Management System (TDMS)
Signal & Telecom	Signals, point machines, track circuits, axle counters, interlocking	Signalling Maintenance & Management System (SMMS)

Corridor availability comes from the **Control Office Application (COA)**. Formal block requests are filed in the **Block & Disconnection Management System (BDMS)**, which remains the system of record.

1.2 Three mechanisms for taking an asset offline

- **Traffic block** — no trains on that line for a defined period. Engineering work: tamping, rail renewal, welding, deep screening.
- **Power block** — OHE de-energised. On an electrified section this implies a traffic block as well, because electric traction cannot run under dead wire.
- **Disconnection** — signalling gear temporarily disconnected from the interlocking, taken with the Station Master's authority. Narrower than a full traffic block.

1.3 Explicit boundaries

Not rolling stock. Coach maintenance, periodic overhauls and shed scheduling belong to the Mechanical / Carriage & Wagon department, operate on different systems, and do not require a block — the coach comes to the shed, and the shed does not stop trains running. None of TMS, SMMS, TDMS, COA or BDMS touches coach maintenance.

Blocks, not possessions. A traffic block is a few hours between trains. A **possession** is a longer planned occupation with the timetable formally modified around it, used for rail renewal campaigns or bridge girder replacement. Possessions are planned months ahead at zonal level. They enter this system as **fixed unavailable windows**, not as tasks to be scheduled.

1.4 Design principles

Principle	Consequence in the design
The system advises; humans sanction	No block is authorised without departmental approval and operations sanction. Full audit trail on every transition.
Never remove existing authority	Sr.DOM keeps final refusal. Departments keep the right to reject. The system supplies information, not decisions.
BDMS stays the system of record	This is a planning layer above BDMS. On conflict, BDMS wins.
Explainability over accuracy	Interpretable models preferred. Every score decomposes into named factors.
Policy in config, not code	Objective weights, freeze horizons, buffers and thresholds are configuration. Divisions differ.
Degrade gracefully	Every task is always available as a singleton candidate. Every model has a rule-based fallback.
Never a surveillance tool	Execution data aggregates at team and section level only, enforced in the API, not just the UI.

2 The network model

2.1 Three levels

Level	Graph role	Contents
Station	Node	Code, km from origin, block station flag, yard flag, route capacity
Block section	Edge	Between consecutive block stations, per line . Where tasks live and where blocks are granted
Corridor	Named path	A run of block sections. Used only for the monthly horizon and human vocabulary

2.2 It is a multigraph

Between two stations on a quadruple-line section there are four edges, not one. They share endpoints and kilometre range but are separately blockable. The edge key is `(from_station, to_station, line_id)`.

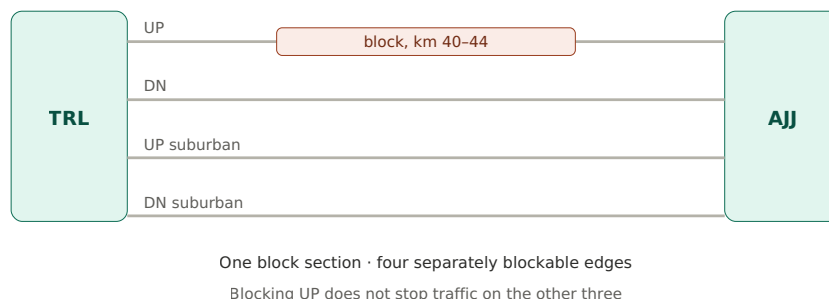


Figure 1 — Parallel edges between two stations. A block occupies one edge.

This is why `line_count` carries so much weight in the cost model. On a single-line section one edge exists and blocking it severs the graph — no train passes. On a quadruple section, blocking UP at 02:00 costs almost nothing because three edges remain.

2.3 Two kinds of adjacency

- **Longitudinal** — the next block section along the route. A train held here piles up there. Gives cascade delay. Ordinary graph traversal.
- **Lateral** — the parallel edges within the same block section. Gives rerouting: when UP is blocked, can traffic work bidirectionally on DN? A property of the edge group, not a traversal.

These are kept as separate relations. Conflating them causes either missed reroute options or double-counted delay.

2.4 Tasks live on edges or on nodes

The resolution question — junctions, crossings, conflict zones — resolves through this split, which mirrors the actual working rules:

Task location	Access mechanism	Model
Edge (mid-section plain line)	Traffic or power block	Edge occupancy. Covers the bulk of block-requiring work: tamping, screening, rail renewal, OHE inspection
Node (station or yard)	Disconnection	Node resource model — a station has a route count, a disconnection consumes some, compatibility rules govern concurrency

A junction such as Arakkonam is simply a node of degree greater than two. Blocking one arm leaves the others traversable, which is correct behaviour for block planning.

Why this granularity is right. A block request is itself a station-to-station document: *block on the UP line between Tiruvallur and Arakkonam, 01:30 to 04:30*. The model's resolution matches the artefact it produces. Finer modelling would require interlocking and route-table data that is not present in TMS, SMMS, TDMS or COA, and would answer a question — minute-by-minute train regulation — that belongs to Control on the day, not to a weekly planner.

2.5 Two refinements that recover most of the lost realism

Adjacent-line speed restriction. When men and machines work on UP, the parallel lines receive a caution order. The cost is charged over the **worksite length**, not the whole block section — roughly one kilometre at 30 km/h instead of 110 gives about 1.4 minutes per train. Applying it over a 26 km section would give 38 minutes per train and swamp the objective function.

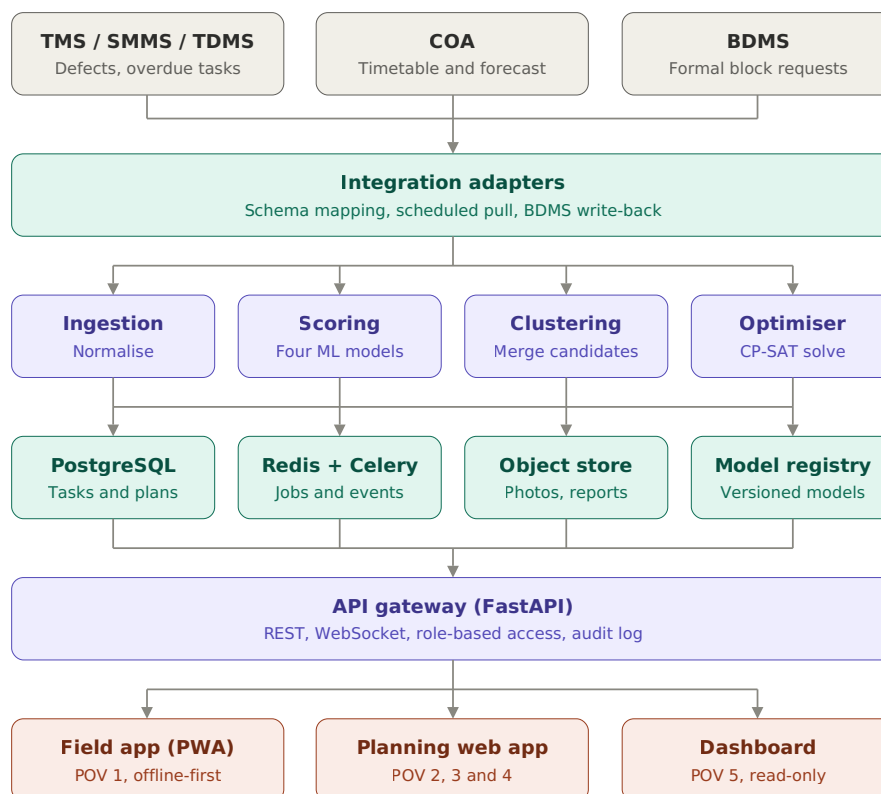
Single-line working is not free. When UP is blocked on a double-line section, trains may work bidirectionally on DN, but capacity falls sharply and crossovers are needed at both ends. Modelled as a capacity factor on the surviving edge, not as costless rerouting.

2.6 Consciously outside the model

- Fouling and route conflicts inside station limits
- Crossover positions, which determine where single-line working can begin and end
- Platform-level allocation
- Signal-spacing block sections in automatic territory — suburban stretches are modelled station-to-station regardless of signalling, matching how blocks are actually requested

None of these change *which day* a block should happen.

3 System architecture



Sanctioned plans return upward through the adapters into BDMS

Figure 2 — Component architecture, six tiers.

Tier	Responsibility
External systems	TMS, SMMS, TDMS supply pending work. COA supplies timetable and traffic. BDMS both supplies existing requests and receives sanctioned plans.
Integration adapters	One module per source. Scheduled pull, schema mapping onto the unified task schema, reconciliation of duplicates from re-inspection. Where most real-world engineering effort goes.
Core services	Ingestion, scoring, clustering, optimiser. Separate modules even within a monolith, so the optimiser can move to a background worker without refactoring.
Data and runtime	PostgreSQL for task pool, plans, approvals and audit. Redis and Celery for queued solves and event-driven re-planning. Object store for photographs. Model registry as versioned artefacts with a metadata table.
API gateway	Single FastAPI service. REST plus WebSocket for live plan updates. Role-based access control is mandatory, not optional. Every state change writes to the audit log.
Clients	Three applications, five points of view. POVs 2, 3 and 4 share one web application under different roles.

Deployment shape. This runs as a **modular monolith**: one FastAPI process containing all four services as packages, one PostgreSQL, one Redis, one Celery worker. The diagram shows logical components, not separate deployments. Service boundaries are drawn but not split, because splitting at this scale adds operational cost for no benefit.

3.1 The two paths through the system

Scheduled path — adapters pull, ingest, score, cluster, optimise, persist, notify. Runs weekly on a cron trigger.

Event path — an emergency is declared in the planning application, the API publishes an event, Celery picks it up, clustering and the optimiser re-run for affected sections only, and the revised plan pushes live to every connected client over WebSocket.

4 The processing pipeline, stage by stage

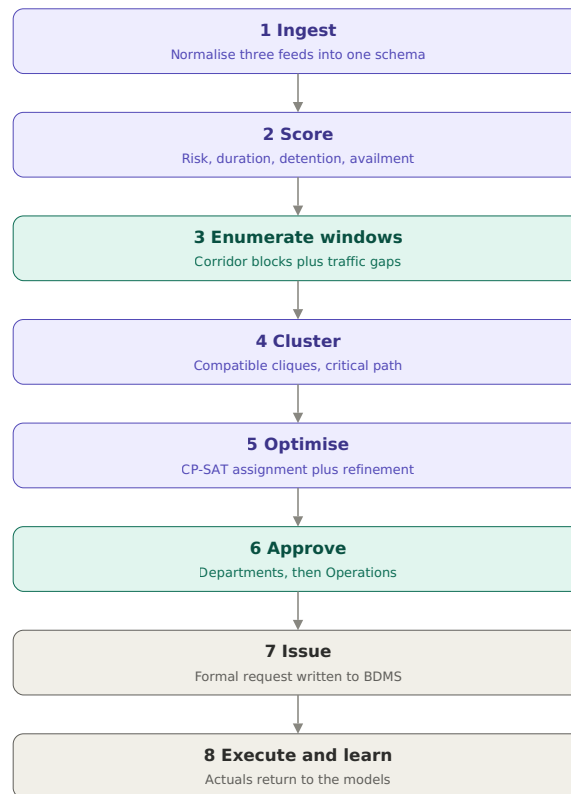


Figure 3 — The eight pipeline stages.

#	Stage	What happens
1	Ingest	Pull from the three departmental systems. Map onto the unified task schema. Geo-tag to block section, km and line. Deduplicate re-inspections of the same defect. Validate and reject malformed records with a reason.
2	Score	Run four models over every pending task: predicted duration with uncertainty, escalation hazard, availment risk. Attach feature attributions for explainability. Compute a composite risk score for officer-facing ranking.
3	Enumerate windows	Read pre-planned corridor blocks. Detect traffic gaps from train paths. Subtract setup and clearance margins. Discard sub-threshold gaps. Emit candidate windows, flagged by type.
4	Cluster	Group pending tasks by block section and line. Enumerate compatible subsets. Compute critical-path duration, access-type union, line union, peak crew and machine requirements. Always emit singletons.
5	Optimise	CP-SAT set-packing: choose (candidate, window) pairs minimising total cost subject to hard constraints. Then a local refinement pass slides each block's start and duration within its traffic gap.
6	Approve	Departmental review by every department with work in the block, then operations sanction. Structured rejection reasons. Block degradation before re-solving.
7	Issue	Sanctioned blocks written to BDMS as formal requests, one per department, cross-referenced by a shared merge group identifier. Freeze horizon now applies.
8	Execute and learn	Field application records readiness, actual start and end, overrun reason, asset status. Actuals return to the models.

5 Stage 3 in detail: window enumeration

5.1 Two sources of windows

Pre-planned corridor blocks — already negotiated into the working timetable, already sanctioned in principle. Read directly.

Traffic gaps — holes in the train pattern wide enough to work in. Detected, not given.

5.2 A gap is not a window

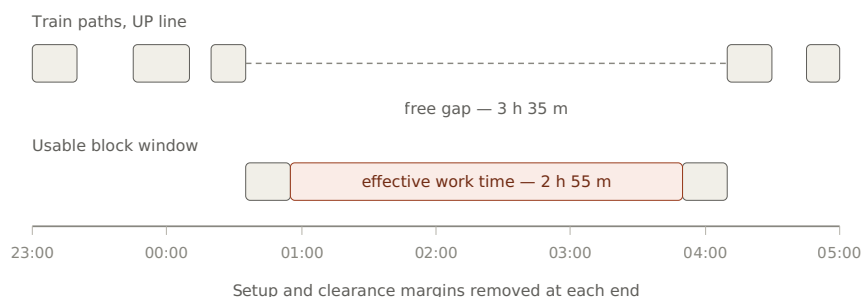


Figure 4 — Setup and clearance margins reduce a raw gap to usable work time.

A block does not begin the moment the last train clears. Men and machines must reach the site; at the other end the line must be cleared, tools accounted for and the section certified fit before traffic resumes. The enumeration algorithm:

```
for each edge (from_station, to_station, line_id), for each day:
    occupations = train paths overlapping this edge
    merge overlapping occupations
    raw_gaps = complement of merged occupations
    for each gap:
        usable = gap - setup_min - clearance_min
        if usable >= min_usable_min:
            emit window(edge, start + setup_min, usable, type = traffic_gap)
    emit all corridor blocks for this edge as type = corridor_block
```

5.3 Three complications, all modelled

Complication	Handling
Goods trains are not in the timetable	Freight is pathed dynamically. A gap that looks empty in the passenger working timetable may be filled by a port-bound rake. The goods forecast converts an apparent gap into a probabilistic one, materially on the northern line.
Both lines cannot be blocked at once	On a double-line section, overlapping gaps on UP and DN may not both be taken — that severs the section. Enforced as a cross-edge constraint at solve time.
Gap availability varies by line	Suburban lines carry EMUs at high frequency all day; gaps exist only after midnight. Trunk lines may have usable mid-morning holes. This asymmetry is real and is surfaced in the corridor board.

5.4 Where corridor blocks come from **RESOLVED**

Corridor blocks are negotiated annually into the working timetable as a fixed pattern per section per day of week. Because the real pattern is not available for this build, it is **generated from a documented rule set** rather than arbitrarily, and the rule set is configuration:

Corridor	Modelled pattern	Rationale
Chennai Central – Arakkonam (quadruple trunk)	2 h per line per week, night slot 01:00–03:00, staggered across sections	Multiple lines allow rotation; night minimises detention on a dense trunk route
Chennai Beach – Tambaram (saturated suburban)	3 h per line, twice weekly, 00:30–03:30	No daytime capacity exists; all maintenance compresses into the night gap
Arakkonam – Kanchipuram – Chengalpattu (single line)	3 h once weekly, mid- morning	Low traffic density means daytime windows genuinely exist; a night block would idle crews for no gain
Basin Bridge – Gummidipoondi (freight)	2 h twice weekly, mid- morning	After the overnight freight peak to Ennore and Chennai Port

Stated limitation. This pattern is a modelled approximation of typical practice, held in configuration. On deployment it is replaced wholesale by the division's actual corridor block table from the working timetable. No claim is made that these are Chennai Division's real corridor blocks.

6 Stage 4 in detail: clustering and merge candidates

6.1 Geography is handled by the grouping key

Because block sections are the unit, no distance calculation is needed for grouping: **same block section plus same line means the same block can cover both**, whether the jobs are 200 metres or eight kilometres apart. Distance re-enters only for specific safety rules carrying a `max_distance_m` condition.

6.2 Clustering is maximal clique enumeration

A valid merge requires **every pair** in the set to be compatible, not merely each with the first. This is clique finding on the compatibility graph — NP-hard in general, irrelevant at this scale: one block section holds perhaps five to twenty pending tasks. Candidate size is capped at five or six, which is realistic; no genuine block carries ten departments.

6.3 Access type is a union, and unions escalate

Tasks in group	Merged block requires
traffic + traffic	traffic block
traffic + power	traffic and power block
traffic + disconnection	traffic block, plus Station Master authority for the node work
power alone, on an electrified section	power block implies a traffic block

Lines behave the same way: the merged block occupies the **union** of its members' lines. Three tasks on UP is cheap; two on UP and one on DN takes both lines and, on a double-line section, severs it.

6.4 Duration is a critical path

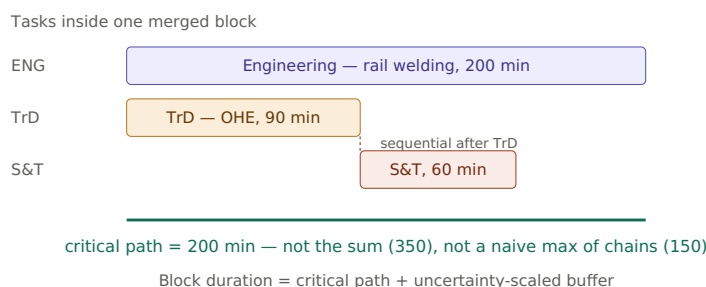


Figure 5 — Merged block duration is the longest path, not a sum and not a naive maximum.

For groups of five or fewer, build the precedence graph from the compatibility relations (`parallel` and `sequential`) and take the longest path. Roughly twenty lines of code, and it is the difference between a plan that holds and one that quietly overruns.

6.5 Candidate generation

```
for each (block_section, line_group):
    tasks = pending tasks on this section
    for each compatible subset up to size 5:      # pruned early via compatibility matrix
        duration      = critical_path(subset)
        access        = union of access types, with escalation rules
        lines          = union of line_ids
        crew_peak      = max concurrent crew across the precedence graph
        machines       = set of machines required
        if infeasible on its own: discard
        emit candidate
    emit every task as a singleton candidate
```

The always-present singletons are what make the optimiser degrade gracefully: if nothing merges, a valid plan still results.

7 Stage 5 in detail: the optimiser

7.1 Paradigm: window filling with refinement

Candidate windows are enumerated in stage 3; the optimiser performs **assignment**, then a local search slides each selected block's start and duration within the traffic gap it occupies. This captures most of the benefit of free-form block construction while remaining a clean set-packing problem that solves fast, debugs easily, and produces output corresponding to the railway's existing planning vocabulary.

7.2 Formulation

```
minimise  Σ detention_cost(c, w)      over selected (candidate, window) pairs
          + Σ deferral_cost(t)        over tasks left unscheduled
          + λ_waste   × unused window minutes
          + λ_access × count of newly requested windows
          + λ_fair    × night-shift imbalance across crews

subject to
  every task covered at most once
  hard deadline on every safety-critical task
  crew capacity and qualification match
  crew rest and duty-hour limits           [hard]
  machine availability and transit feasibility [hard]
  pairwise compatibility within each candidate
  line-union feasibility (never sever a section)
  seasonal and night-work restrictions
  maximum blocks per corridor per week
  predicted duration + buffer fits the window
```

7.3 The objective function in detail

Both sides of the trade-off are measured in **weighted train-delay minutes**. This is what turns “maximise asset availability” into arithmetic.

Cost of taking a block

- **Trains that wanted the edge.** Divertible → small fixed reroute penalty (~8 minutes). Not divertible → held until block end, delay = `block_end - scheduled_entry`. Neither possible → large cancellation penalty.
- **Adjacent-line caution cost** — charged over worksite length, roughly 1.4 minutes per train on parallel lines.
- **First-order cascade** — a held train delays those behind it. One hop only.

Worked comparison — the Tiruvallur block

	Night: Wed 01:30-04:30	Day: Wed 10:00-13:00
Trains on UP	2 goods, both divertible	12; DN saturated, 5 must be held
Held cost	0	5 × 40 min × 1.4 ≈ 280
Reroute cost	2 × 8 × 0.6 ≈ 10	7 × 8 × 1.4 ≈ 78
Caution on parallel lines	6 trains × 1.4 ≈ 8	30 trains × 1.4 ≈ 42
Total weighted minutes	≈ 18	≈ 400

Cost of deferring

```
deferral_cost(t) = ΔP(escalate over the next week)
                  × delay_minutes_per_day_under_speed_restriction
                  × expected_days_until_fixed
```

Two rules that must hold. Use the **marginal** probability for the deferral period, not cumulative — cumulative massively overstates risk and causes panic scheduling. And **safety-critical tasks never participate in this trade-off**: they carry hard deadlines and are constraints, not costs. A probability must never be allowed to weigh against a statutory limit.

Three additional terms

Term	Purpose
Window waste	Penalises unused window capacity. This is what actively pushes the solver toward merging rather than merely permitting it.
New access requested	Soft penalty per newly requested window. Corridor blocks are used first; fresh access is asked for only when it pays. The flagged-access policy lives here, in the cost function, not in a conditional.
Night-shift fairness	Penalises uneven night-work distribution across crews. Cheap, and the difference between a system staff accept and one they resent.

7.4 Priority weights

Delay minutes are weighted by train type. The balanced column is the default, and the weights are **visible in the user interface** so officers can see what the system is valuing.

Train type	Aggressive punctuality	Balanced (default)
Rajdhani class / Vande Bharat	3.0	1.8
Superfast / Express	2.0	1.5
Suburban EMU	1.5	1.3
Passenger	1.0	1.0
Goods	0.3	0.7

The aggressive column mirrors how punctuality is officially measured, but pushed too far it makes freight the dumping ground — every block lands on goods paths and port traffic suffers. That optimises a metric rather than the railway.

7.5 Duration buffering

The duration model returns a distribution. Work is scheduled to the **mean**, and the sanctioned block carries a **buffer scaled by the model's uncertainty** — confident predictions get a small buffer, uncertain ones get more. Overruns are absorbed rather than extending into traffic, and the uncertainty is made visible to officers rather than hidden inside a point estimate.

7.6 Two hard constraints that prevent impossible plans **ADDED**

Constraint	Why it must be hard, not soft
Crew rest and duty hours	Minimum rest between duties, maximum consecutive night shifts, maximum weekly duty hours. Without this the optimiser will assign the same crew a Tuesday night block and a Wednesday morning block. Duty rules are non-negotiable; a plan that breaches them is infeasible, not merely undesirable.
Machine transit feasibility	If a tamper is physically four hours away and the block starts in two, that is impossibility, not risk. Transit time from the machine's current section must fit before block start. Availment risk handles the <i>probabilistic</i> case; this handles the <i>certain</i> one.

8 The four machine learning models

8.1 Duration prediction

Target: actual work minutes, from `actual_end - actual_start`. **Granularity:** per task, not per block — the merged duration is composed by critical-path logic. This also roughly triples usable training rows, since every task contributes whether it ran alone or merged.

Feature source	Features
Task type catalogue	nominal duration, minimum crew, machine class
The task	requested duration, span in metres, severity, materials ready
Asset master	asset age, cumulative tonnage, failures in last 12 months
Section master	line count, sectional speed, traffic type
Crew and machine	crew size, qualification match, machine output rate, transit distance
Context	night or day, season, weather, whether part of a merged block

The `is_merged` feature is not optional. Work inside a merged block is often slower per task — more people on a confined worksite, shared access, coordination overhead. Omitting this feature makes the model systematically underestimate exactly the blocks the system exists to create.

Algorithm: LightGBM **quantile regression** at P50, P80 and P95. Three cheap models; the spread between them is the uncertainty, and the buffer is P80 – P50. Preferred over a single distributional model for interpretability.

Validation: temporal split — train on months 1–9, test on 10–12. Random k-fold leaks future information and flatters results. Report MAE in minutes, and more importantly **overrun rate at the chosen buffer**.

8.2 Escalation risk — survival analysis, not classification

The naive framing — “did this defect escalate?” — is wrong because of **censoring**. A defect attended on day 6 did not escalate, but neither did it get the chance to. Treating it as a clean negative teaches the model that such defects are safe when they were rescued.

Formulation: discrete-time survival. Expand each defect into one row per day it remained open, with the label being whether the escalation event occurred that day. Ordinary binary classification on defect-days; the output is a **daily hazard rate**.

Two things follow directly:

- **Cumulative risk over any horizon** by composing daily hazards — weekly for the weekly plan, 30-day for monthly.
- **The marginal probability the deferral cost requires.** Hazard *is* marginal risk, delivered by construction.

The confounding trap. Dangerous-looking defects are fixed fastest, so in the raw data severe defects rarely escalate. Trained naively, the model learns that severity 1 is safe — exactly backwards. Modelling hazard *conditional on remaining open* mitigates this by asking “given this is still open on day 5, what is today's risk” rather than “does this type escalate”. It does not fully solve the problem; nothing short of a randomised trial would.

Calibration is mandatory because the output feeds a cost function. Isotonic regression on a held-out set, with a reliability curve reported.

8.3 Detention estimator — analytical core with learned residual

```
predicted_detention = analytical_estimate + residual_model(features)
```

The analytical component does the heavy lifting: overlapping trains, held or divertible, reroute penalties, caution costs, one-hop cascade. Transparent, explainable, requires no training data. The residual model learns where that arithmetic is **systematically wrong** — during peak, in known bottleneck sections, when yard capacity at the holding point is tight, or when the reroute path is itself congested.

This framing degrades safely: with no training data the residual is zero and the analytical estimate stands. It also permits a concrete claim — “the analytical model was off by an average of 12 minutes; the residual correction reduces that to 4.”

8.4 Availment risk

Target: binary — was the sanctioned block used at all? **No system in service today models this.**

Features: materials ready, machine current section versus task section, machine availability slack in hours, crew qualification match, lead time between sanction and block date, whether the block was rescheduled from an earlier slot, season and weather, department. The strongest signal is expected to be **machine transit slack**.

Algorithm: LightGBM classifier. Class imbalance makes accuracy meaningless — use precision-recall AUC, and calibrate so a predicted 0.3 genuinely means 30%.

Use in the optimiser: as a **cost multiplier** scaling expected block value, **plus a hard cutoff at 0.6**. Below the cutoff it is a trade-off; above it, the block should not be proposed at all — the optimiser is forced to fix the cause rather than gamble.

8.5 Cold start — three stages, all shipped

Stage	Data available	Behaviour
1 — Rule-based	None	Duration = requested + fixed buffer by task type. Availment = base rate from the catalogue. Fully transparent.
2 — Hierarchical fallback	Under ~200 executions	Task-type mean where samples suffice, backing off to department mean, then global mean. Blended with the rule-based estimate, weighted by sample count.
3 — Full models	Sufficient	Trained models, with the rule-based version retained as a monitored fallback should performance degrade.

The system works from day one and improves, rather than requiring a training set before it can run.

8.6 Scope of escalation modelling

Per-defect is the core. Only reported defects carry risk; the system schedules work someone has flagged. A **per-asset secondary layer** surfaces proactive inspection candidates — assets with no open defect but high tonnage, age and failure history — feeding the same task pool but visibly marked as system-generated. Officers decide what enters the pool.

9 Multi-horizon planning

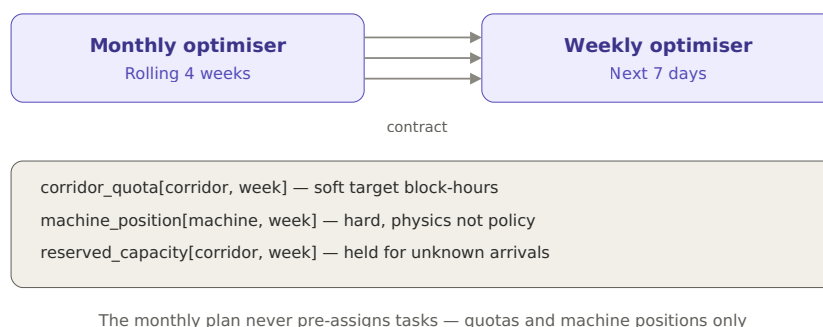


Figure 6 — The monthly-to-weekly handoff contract.

9.1 Why one planner cannot do both

Running the weekly optimiser over 30 days grows the problem to roughly 1,700 tasks with worse-than-linear candidate enumeration. More importantly the answer would be **fiction**: assigning a specific task to a specific Tuesday 27 days out, when the defects of week three have not been raised, the tamper's position is unknown, and emergencies have not yet consumed windows. Precision without information is not planning.

9.2 Different decisions at different horizons

	Monthly	Weekly
Question	Where should capacity go?	What happens when?
Unit	Corridor × week	Task × window
Output	Block-hour quotas, machine positioning	Sanctionable block plan
Assumes	Forecast task arrivals	Known pending tasks
Re-runs	Weekly, rolling 4-week window	Weekly

9.3 Machine positioning is what makes monthly planning necessary

A tamper cannot teleport. If week 3 carries heavy tamping demand on the Arakkonam corridor, the machine must begin moving in week 2 — a decision the weekly planner cannot make, because by the time week 3 runs the machine is already in the wrong place. The monthly optimiser assigns each scarce machine a sequence of corridor-weeks, minimising transit time and unmet demand. The same logic applies to materials with long lead times: rails, turnout assemblies, OHE components.

9.4 Forecast demand

- **Deterministic** — scheduled periodic maintenance. Every asset with a periodicity produces a predictable task. This is the majority, and requires no machine learning.
- **Stochastic** — defect arrivals, per section per asset type. A Poisson rate estimated from history is sufficient.

The key move. The monthly plan does not schedule unknown defects — it **holds capacity open for them**. A corridor with a high defect arrival rate receives more block-hour quota than its known backlog justifies. Fill every hour with known work and week three's emergencies have nowhere to go.

9.5 The handoff contract

```
corridor_quota[corridor, week]    → soft target block-hours  
machine_position[machine, week]   → where the machine will be  
reserved_capacity[corridor, week] → capacity held for unknown arrivals
```

Quotas are soft. If a week's actual backlog exceeds its quota, the weekly planner exceeds it with a penalty rather than refusing — the monthly plan is guidance from an information-poor view, and the weekly planner has better information. **Machine positions are hard** — physics, not policy.

The trap. The monthly plan must **never pre-assign tasks**. The temptation is to run a coarse assignment and hand week 1 to the weekly planner as a starting point. That bakes in decisions made without week 1's actual information and burdens the weekly planner with constraints it should be free to reject. Quotas and machine positions only.

9.6 Implementation

The monthly optimiser is a **separate CP-SAT model**, roughly 60 lines against the weekly planner's several hundred. Its variables are genuinely different — corridor-week quotas and machine-week assignments, not task-window assignments — and forcing an assignment formulation to express a quota problem costs more in contortion than it saves in code.

Quota adherence is tracked: how far actual weekly usage drifted from monthly guidance. Consistent overshoot means the forecast underestimates demand, which is diagnostic information officers would genuinely act on.

10 Emergency re-planning

10.1 The governing reframe

An emergency is not an input to the planner. It is an input to reality. By the time the system hears about it, the section is occupied, the crew is diverted, and nobody consulted an algorithm. The system's question is never “should we respond” but “given that this has happened, what is the best plan for what remains?”

10.2 What an emergency consumes

Consumed	Recovery profile
The section	For the incident duration, usually hours. Recovers on its own.
The resources	The crew and machine diverted to attend. Usually the larger disruption — a rail fracture near Ennore pulls the P.Way crew that was to do Wednesday's welding at Tiruvallur. That section is completely fine; the work still cannot happen.
Trust in the schedule	Everything downstream becomes provisional until re-planned.

10.3 The freeze horizon



Figure 7 — Three zones governing what the optimiser may move.

Zone	Window	Behaviour
Frozen	0–48 hours	Issued to Control, crews briefed, materials moved. The optimiser cannot move these. Only a human cancels.
Soft	48 h – 7 days	Approved but not yet issued. Re-optimisable, but changes are flagged as revisions requiring re-approval.
Open	Beyond 7 days	Freely re-optimised, no notification.

The critical exception: a frozen block whose resources were consumed by the emergency is now **infeasible, not suboptimal**. It is not re-optimised — it is **cancelled and surfaced to the officer with the reason**. The system never silently reschedules a committed block.

10.4 The re-plan sequence

1. **Declare** — Control logs the event: type, section, km, line, estimated duration, resources diverted.
Manual declaration — only Control knows the true duration estimate and which crew was actually diverted.
2. **Invalidate** — mark affected windows unavailable; mark resources unavailable for incident duration plus recovery and transit; identify now-infeasible blocks.
3. **Classify** — frozen and infeasible → cancel and escalate. Soft → return tasks to the pool. Open → return silently.

4. **Re-solve** — **automatically**, over the open and soft horizons, with frozen blocks as fixed constraints occupying their windows and resources.
5. **Diff** — compute what changed from the previous plan.
6. **Notify** — affected users only.

Present a diff, not a new plan. “Three blocks moved, one cancelled, two unchanged” is comprehensible at 03:00. A regenerated full schedule is not, and an officer who cannot see what changed will not trust it.

10.5 Why manual declaration and automatic re-plan

Auto-detection from departmental feeds sounds impressive but risks a data glitch cancelling real blocks. “The system cancelled tonight's work because of a bad feed” is exactly the failure that gets a tool switched off permanently. A human confirms and scopes the event; the machine does the recomputation instantly.

10.6 Cascade and its trap

Emergency work generates new tasks. A rail fracture receives a temporary repair overnight, but the permanent replacement becomes a fresh high-priority task with a short hard deadline, flagged `origin: emergency_followup`, competing for the same scarce windows.

Watch for crowding out. In a high-emergency simulation, follow-up tasks can starve routine work indefinitely — the backlog quietly explodes while detention figures look excellent. This is a genuine failure mode of priority-driven scheduling, not a coding bug, and must be monitored in evaluation.

10.7 Performance

The re-solve is fast because it is scoped: frozen zone fixed, restricted to affected sections and their neighbours. A CP-SAT time limit applies — a good-enough plan in two seconds beats an optimal one in forty when responding to disruption. Results push over WebSocket to connected clients.

11 Approval workflow and BDMS handoff

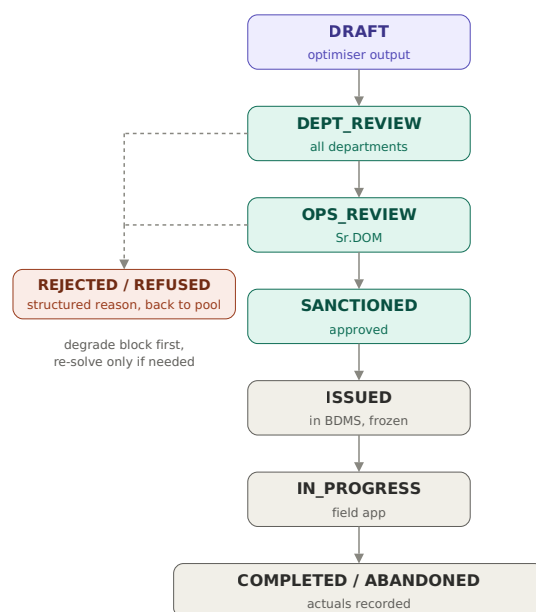


Figure 8 — The block lifecycle state machine.

11.1 States

State	Meaning
DRAFT	Optimiser output. Nobody has seen it.
DEPT_REVIEW	Requires approval from every department with work in the block. A merged ENG+TrD+S&T block needs three approvals — genuinely new, because those officers currently never share a block.
OPS_REVIEW	Sr.DOM sees traffic impact and holds final authority.
SANCTIONED	Approved, not yet formally requested.
ISSUED	Written to BDMS as a formal request. Point of no return; the freeze horizon now applies.
IN_PROGRESS / COMPLETED / ABANDONED	Execution states driven by the field application.

11.2 Partial approval — the hard case

Three departments must approve; two do; one rejects. The naive responses are both wrong: cancelling the whole block punishes cooperating departments for someone else's problem, and they will stop trusting merged blocks.

The design: degrade first. Drop the rejecting department, recompute critical path and cost. If the reduced block still beats the alternatives, keep it and notify the remaining departments that scope changed. If it does not, return everything to the pool and re-solve.

Approvals survive unchanged scope. If Engineering approved a three-hour block and it remains a three-hour block for Engineering, do not make them approve again. Re-approval fatigue is how workflow systems get abandoned.

11.3 Structured rejection

Rejection reasons are an enumeration, not free text, for three reasons:

- **Model feedback** — `machine_late` rejections are training signal for availment risk. Free text is unusable.
- **Automatic remediation** — `duration_insufficient` triggers a re-solve with a longer requirement; `crew_unavailable` re-solves excluding that crew. The system proposes an alternative immediately rather than returning the problem to the officer.
- **Measurability** — rejection rate by reason and by department over time is exactly the diagnostic a DRM would want.

Free text is permitted *in addition*, never instead.

11.4 Approval policy

Work type	Policy
Safety-critical, or newly requested access	Explicit approval required. No auto-approve.
Routine work inside existing corridor blocks	Timed auto-approve after 48 hours, with escalation notices at 24 and 36 hours.

Risk is proportionate to what is being decided; the common case flows without silently committing anyone to something consequential.

11.5 BDMS boundary

Operation	Behaviour
Read	Existing block requests already in BDMS, so windows claimed outside this tool are not double-booked.
Write	Sanctioned blocks as formal requests. A merged block is issued as one request per department, cross-referenced by a shared <code>merge_group_id</code> — fitting existing structures while preserving the merge information.
Never	Bypass BDMS, or treat this system's database as the source of truth for block authorisation.
On conflict	BDMS wins. A block present there that this system did not create marks that window occupied.

11.6 Audit

Every transition records who, when, from-state, to-state, reason, **the plan version, and the optimiser's cost figures at the moment of decision**. If something goes wrong months later, the question will not be what the system decided but **what information the officer had when they approved it**. Safety-critical systems must be able to answer that.

11.7 Persistent rejection feedback

A proposal rejected three or more times is flagged to the DRM by name. Repeated rejection means either the compatibility matrix is wrong or a real constraint exists that nobody encoded. This is the mechanism by which the system learns things it was never told.

12 Evaluation harness

12.1 The problem: no ground truth

Chennai Division cannot be run for a month with the system and a month without it. The claim must therefore come from a **counterfactual baseline** — simulate current practice and the coordinated planner on identical inputs, and measure the difference.

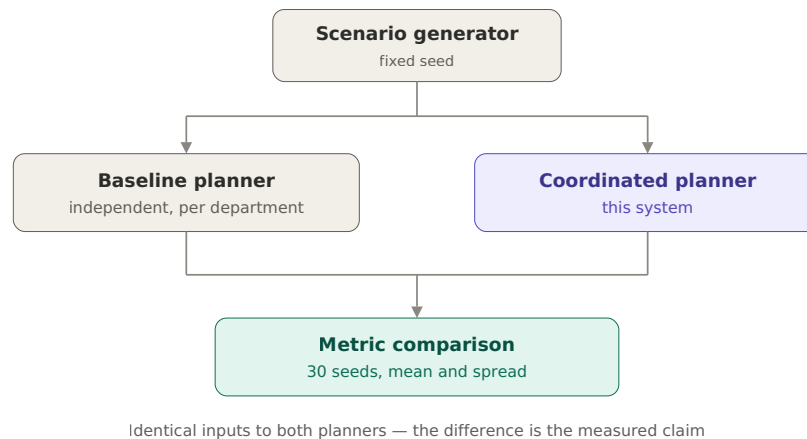


Figure 9 — Both planners receive identical scenarios; the difference is the measurement.

12.2 The baseline must be fair

Current practice, modelled honestly:

1. Each department independently sorts its own backlog by urgency
2. Requests corridor blocks for its top items, first-come-first-served against availability
3. No visibility into other departments' requests
4. Operations grants what fits, rejects clashes
5. Rejected work rolls to the following week

Note what this baseline is **not**: random, or stupid. Each department behaves sensibly within its own information. **The gap being measured is the cost of missing information**, not the cost of incompetence — a framing that is both true and far more persuasive.

One fairness refinement: an **incidental merge rate of 10-15%**, because informal coordination does happen through phone calls and corridor meetings. This shrinks the headline number and makes it credible.

12.3 Metrics

Tier	Metric	Purpose
Primary	Line occupation hours	The headline claim, aggregated
	Weighted detention minutes	The operational cost
	Merge rate	Blocks containing more than one department
Secondary	Tasks completed	Guards against gaming — a planner scheduling nothing has perfect detention
	Backlog at horizon end	Is the queue shrinking?
	Safety-critical deadline breaches	Must be zero. If not, no other number counts
	Mean days-to-attend by severity	Is urgent work genuinely going first?
Tertiary	Block utilisation	Sanctioned minutes versus worked minutes
	Overrun rate at chosen buffer	Is the duration model calibrated?
	Re-plan recovery time	Responsiveness after an emergency

12.4 Experimental design

- **Thirty independent weeks** with different seeds. Report mean and spread. A single simulated week proves nothing.
- **Stratify by corridor.** Saturated suburban and quadruple trunk behave very differently; merging matters far more where windows are scarce.
- **Ablations** — each one config flag and a re-run:

Component removed	Expected effect
Merging (singletons only)	Isolates the coordination gain specifically
ML duration (use requested)	Overrun rate should climb
Availment risk	Wasted blocks should rise
Detention weighting (any valid window)	Detention should spike

- **Weight sensitivity sweep.** If the gain exists only at one particular goods-train weight, it is an artifact.

12.5 The honesty statement

Results are on synthetic data with parameters chosen by the team. The baseline is a model of current practice, not measured current practice. The compatibility matrix is unvalidated by railway officers. **Absolute numbers are illustrative; the relative comparison is the claim.** The assertion is not “Chennai Division will save 14 hours” but “under identical conditions, coordinated planning dominates independent planning, and here is by how much and why.”

12.6 The harness is built early, not last

Both planners plug into one harness. Every optimiser change is measured immediately rather than judged by eye, and demonstration figures emerge from the same pipeline used during development — no last-minute surprises.

13 Configuration parameter reference

All of the following are configuration, not code. Divisions differ, and being able to state that these are tunable policy parameters rather than hardcoded assumptions is a substantive answer to “who decided the trade-offs?”

Parameter	Default	Governs
Window enumeration		
<code>setup_min</code>	20	Time to get men and machines on track after the last train
<code>clearance_min</code>	20	Time to clear, account for tools and certify the line fit
<code>min_usable_min</code>	60	Below this a window is discarded
<code>corridor_block_pattern</code>	see §5.4	Modelled corridor block table, replaced on deployment
Clustering		
<code>max_candidate_size</code>	5	Maximum tasks in one merged block
<code>buffer_quantile</code>	P80	Quantile used for block buffering
Objective weights		
<code>train_priority_weights</code>	balanced	Delay weighting by train type
<code>lambda_waste</code>	tunable	Penalty on unused window capacity — drives merging
<code>lambda_access</code>	tunable	Penalty per newly requested window — the flagged-access policy
<code>lambda_fair</code>	tunable	Night-shift imbalance penalty
<code>reroute_penalty_min</code>	8	Fixed cost of diverting a train to a parallel line
<code>cancellation_penalty</code>	large	Cost when a train can neither be held nor diverted
<code>caution_worksite_km</code>	1.0	Length over which adjacent-line speed restriction applies
<code>single_line_capacity_factor</code>	0.4	Surviving-edge capacity under single-line working
Constraints		
<code>min_rest_hours</code>	8	Minimum rest between crew duties
<code>max_consecutive_nights</code>	3	Consecutive night shifts per crew
<code>max_weekly_duty_hours</code>	configurable	Weekly duty ceiling per crew
<code>availment_hard_cutoff</code>	0.6	Above this risk, the candidate is not proposed
<code>max_blocks_per_corridor_week</code>	configurable	Practical ceiling on corridor disruption
Horizons and workflow		
<code>freeze_horizon_hours</code>	48	Committed zone the optimiser cannot touch
<code>soft_horizon_days</code>	7	Revisable-with-notification zone
<code>monthly_window_weeks</code>	4	Rolling monthly planning horizon
<code>auto_approve_hours</code>	48	Timed auto-approve for routine work only
<code>persistent_rejection_threshold</code>	3	Rejections before flagging to the DRM
Solver		
<code>weekly_solve_time_limit_s</code>	60	CP-SAT ceiling for the scheduled run
<code>replan_solve_time_limit_s</code>	2	CP-SAT ceiling for emergency response
<code>random_seed</code>	fixed	Reproducibility across development and demonstration

14 Technology stack

Layer	Choice	Rationale
Language	Python 3.11	OR-Tools, LightGBM and the data stack all live here
API	FastAPI	Async, WebSocket support, Pydantic validation matching the task schema
Optimiser	OR-Tools CP-SAT	Set-packing is its natural shape; excellent at this scale
Graph	NetworkX MultiDiGraph	Native support for parallel edges keyed by line
ML	LightGBM + SHAP	Categorical handling, no scaling, seconds to train, feature attribution for explainability
Database	PostgreSQL	Task pool, plans, approvals, audit log
Queue	Redis + Celery	Solves exceed HTTP timeouts; emergency re-plan is event-driven
Frontend	React + D3 scales	D3 for the maths, React for the DOM — never let D3 manipulate the DOM directly
Field app	PWA + IndexedDB	Offline-first, no app store distribution

Closing. The current process is decentralised, manual, and designed for a network with slack. The slack is gone. This system does not replace the people who plan blocks or the systems they already use — it gives them a shared, scored, costed view of work that is presently invisible across departmental boundaries, and proposes plans they remain free to reject. The measurable outcome is hours of corridor time returned to train operations, with no maintenance task sacrificed.